

STATEMENT OF WORK

ReceiptLedger — Software Project Management (SPM-458)

COURSE
SPM-458

SEMESTER
4th

INSTRUCTOR
Humera Azam

UNIVERSITY OF KARACHI • COMPUTER SCIENCE DEPARTMENT

Software Project Management

ReceiptLedger

Turning a phone photo of any paper receipt into structured, categorized spending data.

SUBMITTED BY

#	NAME	ROLL NUMBER	ROLE
001	Aazmeer Sarfaraz Faridy	B24110006002	Member
002	Arish Ahmed Khan	B24110006026	Member
003	Iman Hussain	B24110006054	Member
004	Muhammad Asad Khan	B24110006087	Team Lead
005	Syed Aun Shamsi	B24110006134	Member
006	Zainab Hashmi	B24110006162	Member

1 Introduction

This Statement of Work builds on the approved ReceiptLedger Project Charter, converting its objectives into a concrete technical design, sprint plan, toolchain, and measurable acceptance criteria. It also fulfills Assignment #2's Comprehensive SPM Exercise: Section 4 addresses **Task 2** (PMI Knowledge Areas), Section 5 addresses **Task 3** (Lifecycle Relationships), this document as a whole is **Task 5** (Project Charter & SoW), and Section 11 addresses **Task 6** (PMI Process Groups); Tasks 1 and 4 are addressed directly below.

Every tool and service named in this SoW is free at the scale this project needs — a deliberate constraint given the team is a **six-member, single-semester, six-sprint student group with no project budget**.

Task 1 — SPM Tool

The team's assigned SPM tool is **Trello** (Section 9.2), used for backlog, board, and task tracking across all six sprints.

Task 4 — Classic Mistakes Identified & Avoided

Classic Mistake	How This Plan Avoids It
Product: gold-plating / undefined scope	Section 2 fixes explicit in/out-of-scope boundaries carried from the Charter
Process: unrealistic schedule set before the work is understood	Section 5.3 keeps Sprints 5–6 deliberately unassigned to specific stretch features, decided from real Sprint 1–4 data instead of guessed upfront
People: unclear ownership on a team of this size	Section 10 assigns a Primary Focus per area, with the Lead reviewing across areas rather than owning every ticket
Technology: chasing a trained ML model before validating a simpler approach	Section 6 uses rule/dictionary correction over a trained model, validated against real collected data before anything heavier is considered

2 Project Scope

Carried forward from the Charter; every design decision in this SoW is scoped against these lines.

2.1 In Scope

- Phone-camera capture (single or multi-shot) via a Flutter app.
- OCR of both printed (organized retail) and handwritten (kiriyana/small-shop) receipts.
- Normalization of item-name variants to one canonical item and unit.
- Rule-based categorization into a local spending taxonomy.
- Duplicate-receipt detection via image hashing and text similarity.
- A month-over-month, category-wise spending dashboard (Section 3).

2.2 Out of Scope

- Bank/card reconciliation, tax filing, or invoice generation.
- Categorization beyond common local household/shop keywords.

- Training a custom OCR/handwriting model from scratch.

3 Dashboard — What It Displays

Fixed directly against Objective O4 (clear spending visibility):

View	What it shows
Monthly overview	Total spend for the selected month, with delta vs. previous month
Category breakdown	Spend split by category as a chart, for the selected month
Month-over-month trend	Multi-month view per category
Receipt log	Searchable list of receipts with line items, store, date, category
Review queue	Low-confidence OCR items (Section 6) surfaced for quick manual confirmation

4 PMI Knowledge Areas — Task 2

Applied to ReceiptLedger, mapped to where each is concretely addressed in this SoW:

Knowledge Area	Applied in
Scope	Section 2
Schedule / Time	Section 5.3 (Sprint Breakdown)
Cost	Section 8 (Budget)
Quality	Section 7 (Acceptance Criteria)
Human Resource	Section 10 (Team & Roles)
Communications	Section 9 (Tooling — Slack, Trello)
Risk	Section 12 (Risks & Mitigations)
Stakeholder	Charter Section 06, carried forward unchanged
Integration / Procurement	Not separately applicable at this project's scale — no external vendors or contracts

5 SDLC Methodology — incl. Task 3 (Lifecycle Relationships)

5.1 Framework: Scrum

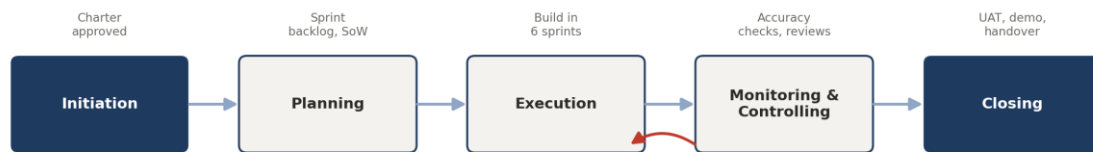
The Charter's six-sprint cadence implies an iterative framework. Scrum fits specifically because:

Factor	Why it points to Scrum
Unproven OCR accuracy going in	Accuracy is measured mid-project (Sprints 2–3) and later sprints re-planned around real results

Factor	Why it points to Scrum
Six loosely-coupled subsystems	Each can be demoed as a working slice — a natural fit for sprint-sized increments
Fixed deadline, flexible feature set	The backlog is trimmed to protect the deadline, not quality
Small, single-room team	Lightweight ceremonies suit six people without process overhead

5.2 Project Lifecycle (Task 3)

The stages below, their dependencies, and what each hands to the next:



Monitoring & Controlling loops back into Planning/Execution each sprint (Scrum)

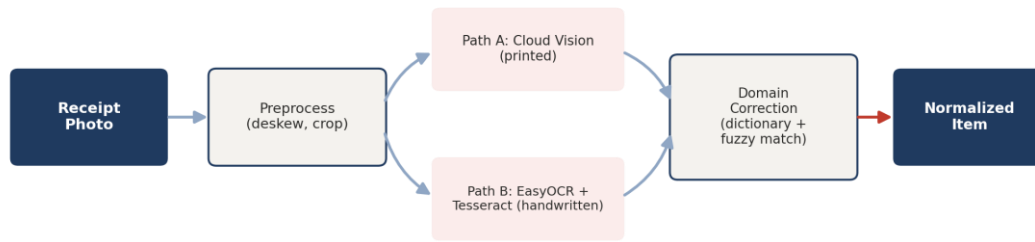
5.3 Sprint Breakdown

Each sprint states its focus and exit criteria. Later sprints are deliberately left flexible — what fills Sprints 5–6 is decided from the backlog based on how earlier sprints land, not fixed months in advance:

Sprint	Focus	Exit Criteria
1	Setup, capture & data collection	Repo, CI, Trello board live; capture screen working end-to-end; real receipt collection underway
2	OCR — printed-receipt path	Printed OCR integrated and accuracy-tested; checkpoint on whether Sprint 3 needs more handwriting time
3	OCR — handwritten path + normalization v1	Handwritten path integrated and tested; normalization v1 working; Sprint 4 scope adjusted from results
4	Categorization + duplicate detection	Rule-based categorizer and dedup live and tested against the accumulated sample
5	Dashboard + integration hardening	Dashboard live with Section 3's views; end-to-end flow stable
6	Testing, polish, demo prep	Bug-fix pass, UAT against Section 7, final demo and documentation

6 OCR Architecture

The highest-risk part of the project: reliable extraction, especially from handwriting, without heavy compute. ReceiptLedger splits the problem into **one capture flow feeding two extraction paths** that reconverge at normalization, rather than one engine trying to solve everything.



Low-confidence items are flagged to the dashboard Review Queue instead of silently accepted

OCR pipeline — printed and handwritten paths reconverge at correction

6.1 Path A — Printed Receipts

- **Google Cloud Vision API** (TEXT_DETECTION), per the Charter. Free tier: 1,000 units/month. Expected accuracy ~95–99% on clean printed text.
- **Fallback:** Tesseract 5 (self-hosted) if the free-tier quota runs out.

6.2 Path B — Handwritten Receipts

Public benchmarks put free/open handwriting OCR at roughly 70–90% word accuracy — the pipeline assumes raw output will contain errors and corrects in layers:

- **Preprocessing:** deskew, binarize, crop (OpenCV) before OCR runs.
- **Engines:** EasyOCR (primary, handles handwriting, CPU-runnable) plus Tesseract 5 as a second opinion for diffing low-confidence tokens.
- **Domain correction:** a rule/dictionary layer (not a second trained model) — OCR output fuzzy-matched against a vocabulary of common local items built from the team's own collected receipts; numeric price/quantity fields get stricter regex checks; anything still low-confidence is flagged to the dashboard's Review Queue (Section 3) instead of silently accepted.

7 Acceptance Criteria

Mapped directly to the Charter's SMART objectives, tested against the team's real collected sample:

Objective	Acceptance Criterion
O1 — Accurate extraction	≥90% accuracy on printed receipts; ≥70% on handwritten post-correction, both manually verified
O2 — Categorization	≥85% of items correctly categorized without manual correction
O3 — Duplicate detection	Zero false negatives on intentionally re-photographed test receipts; low false-positive rate
O4 — Spending visibility	Dashboard totals match manually-tallied totals for the full test dataset

Handwritten accuracy (70%) is set lower than printed (90%) deliberately — an honest target keeps O1 testable rather than aspirational.

8 Budget

Every component in this SoW runs on a free tier or is open-source and self-hosted. The only realistic costs are small and occasional, not structural:

Item	Estimate
OCR API overage (if Cloud Vision free tier is exceeded)	PKR 0–3,000 (Tesseract fallback avoids this entirely)
Mobile data for field data collection	PKR 1,500
Printing & submission binding	PKR 1,000
Contingency buffer	PKR 2,000
Estimated Total	PKR 4,500–10,000

No paid infrastructure, licenses, or subscriptions are required to reach a working, demoable product within the semester — consistent with the Charter's original budget estimate.

9 Project Tooling

9.1 Communication: Slack

- A pinned onboarding message covering channel purpose, set up on day one.
- Free tier covers the semester's message volume and history needs.

9.2 SPM Tool: Trello

Trello (free plan — up to 10 collaborators and 10 boards per Workspace, unlimited cards) is the team's SPM tool for backlog, task tracking, and sprint-style delivery.

Why Trello

- Zero-configuration board model — fastest to onboard a team where nobody has used an SPM tool before.
- Free tier comfortably covers a six-person team (well under the 10-collaborator cap) at no cost.
- Butler automation (250 runs/month free) can auto-move cards, notify on due dates, and archive completed work — covers this team's automation needs without a paid tier.
- **Honest limitation:** Trello has no native sprint/burndown view, unlike JIRA. This is addressed structurally, not ignored (below).

How the team uses it as a sprint board

- One board per project, with one list per sprint status: Backlog → Sprint 1 → Sprint 2 → ... → Done — sprints are modeled as lists rather than a built-in sprint feature.
- Labels for subsystem area (Backend, Flutter, OCR, Dashboard, QA) so the board stays readable at a glance.
- Checklists on each card for a shared “Definition of Done” (code merged, tested against real data, demoed).
- One short onboarding session before Sprint 1 — Trello's board model needs less explanation than JIRA's, so this stays brief.

9.3 iOS Testing: Codemagic

Building/signing an iOS app requires a Mac regardless of Apple account tier, and no team member has one. Codemagic's free tier (500 build minutes/month on cloud Mac instances) builds and signs iOS apps without team-owned Mac hardware, using free Personal Team Apple ID signing — no \$99/year developer fee.

- **Realistic limit:** Mac minutes are the scarce resource — fine for periodic test builds before a demo, not continuous CI. Android stays the primary day-to-day build/test target; iOS builds run on-demand when there's something worth testing on a teammate's iPhone.

10 Team & Roles

Member	Primary Focus
Muhammad Asad Khan (Lead)	Backend architecture, OCR pipeline, sprint coordination
Aazmeer Sarfaraz Faridy	Frontend and UI mockups
Arish Ahmed Khan	Cloud-based deployment
Iman Hussain	Documentation and reports
Syed Aun Shamsi	React dashboard, QA/testing
Zainab Hashmi	Data collection coordination

11 PMI Process Groups — Task 6

Each Process Group applied to ReceiptLedger's delivery, run by the whole team rather than one sub-group per phase — appropriate at this team size, where phases overlap rather than hand off cleanly:



Each Process Group is owned by the whole team, not a single sprint — Monitoring & Controlling runs alongside Executing throughout, consistent with the team's Scrum-based delivery plan

Process Group	Activity, Decisions & Challenges
Initiating	Charter approved; problem, objectives, and stakeholders confirmed (pre-Sprint 1)
Planning	This SoW; Trello backlog built; architecture and tool decisions locked (Sprint 1)
Executing	Build work across Sprints 1–5, per Section 5.3
Monitoring & Controlling	Accuracy checkpoints (Sprints 2–3), sprint reviews, backlog re-scoping — continuous, Sprints 2–6
Closing	UAT against Section 7, final demo, documentation handover (Sprint 6)

12 Key Risks & Mitigations

Risk	Mitigation
Handwritten OCR accuracy falls short even after correction	Sprint 3 accuracy checkpoint; review-queue confidence threshold tightened rather than blocking the sprint
Free-tier quota exhaustion (Cloud Vision, Supabase, hosting, Codemagic)	Tesseract fallback for OCR; usage monitored; iOS builds run on-demand, not continuously
Trello/Slack onboarding friction in Sprint 1	One dedicated onboarding session for each before Sprint 1 begins
Trello's lack of native sprint tooling causes drift from the sprint plan	Sprints modeled explicitly as board lists (Section 9.2) with Definition-of-Done checklists per card

13 Sign-off

This Statement of Work is submitted as the execution plan for the ReceiptLedger project, built on and consistent with the approved Project Charter, for review under SPM-458.

Humera Azam

Course Instructor, SPM-458

Signature: _____ Date: _____